Week 11

Question 1

Write a Java program to read characters from the keyboard using Reader and display them on the console using Writer.

Program (ReaderWriterDemo.java):

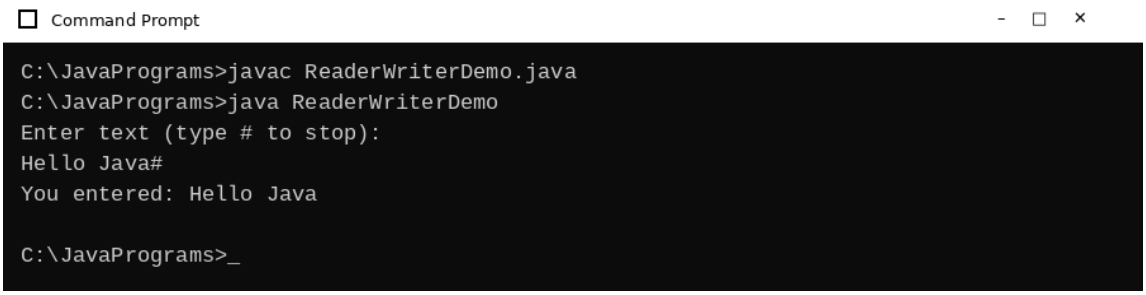
```
import java.io.*;

public class ReaderWriterDemo {
    public static void main(String[] args) throws IOException {
        Reader reader = new InputStreamReader(System.in);
        Writer writer = new OutputStreamWriter(System.out);

        System.out.println("Enter text (type # to stop):");
        writer.write("You entered: ");

        int ch;
        while ((ch = reader.read()) != '#') {
            writer.write(ch);
        }
        writer.write("\n");
        writer.flush();
    }
}
```

Output:



Question 2

Write a Java program to read the contents of a text file using FileReader and display them on the console.

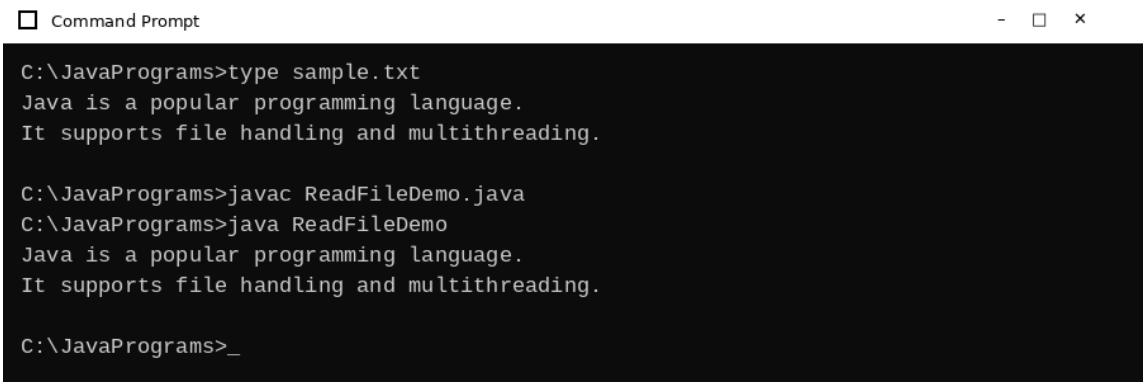
Program (ReadFileDemo.java):

```
import java.io.*;

public class ReadFileDemo {
    public static void main(String[] args) throws IOException {
        FileReader fr = new FileReader("sample.txt");

        int ch;
        while ((ch = fr.read()) != -1) {
            System.out.print((char) ch);
        }
        fr.close();
    }
}
```

Output:



Question 3

Write a Java program to write text into a file using FileWriter.

Program (WriteFileDemo.java):

```
import java.io.*;

public class WriteFileDemo {
    public static void main(String[] args) throws IOException {
        FileWriter fw = new FileWriter("output.txt");

        fw.write("Welcome to Java File Handling.\n");
        fw.write("This text is written using FileWriter.");
        fw.close();

        System.out.println("Text written to output.txt successfully.");
    }
}
```

Output:



Question 4

Write a Java program to copy the contents of one text file into another using FileReader and FileWriter.

Program (CopyFileDemo.java):

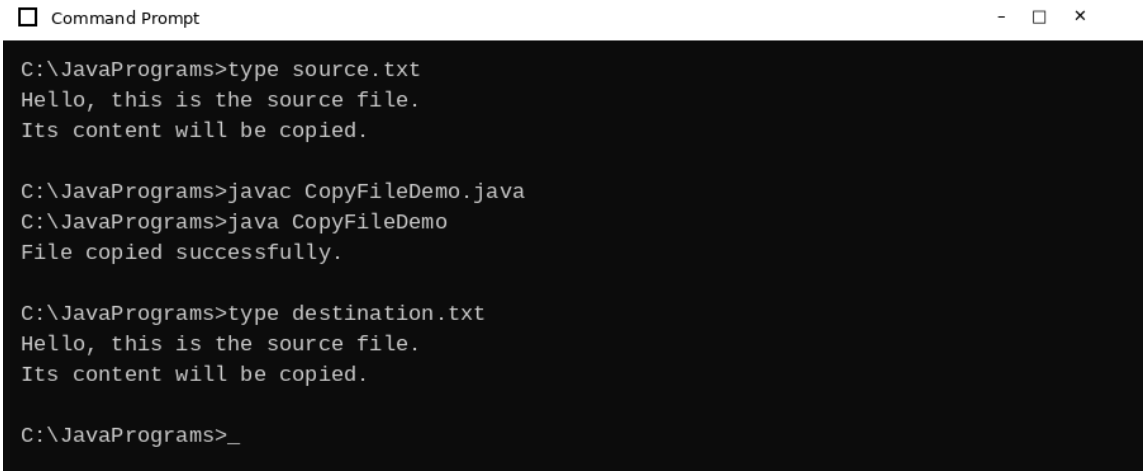
```
import java.io.*;

public class CopyFileDemo {
    public static void main(String[] args) throws IOException {
        FileReader fr = new FileReader("source.txt");
        FileWriter fw = new FileWriter("destination.txt");

        int ch;
        while ((ch = fr.read()) != -1) {
            fw.write(ch);
        }
        fr.close();
        fw.close();

        System.out.println("File copied successfully.");
    }
}
```

Output:



Question 5

Write a Java program to count the number of characters, words, and lines in a text file using character streams.

Program (CountDemo.java):

```
import java.io.*;

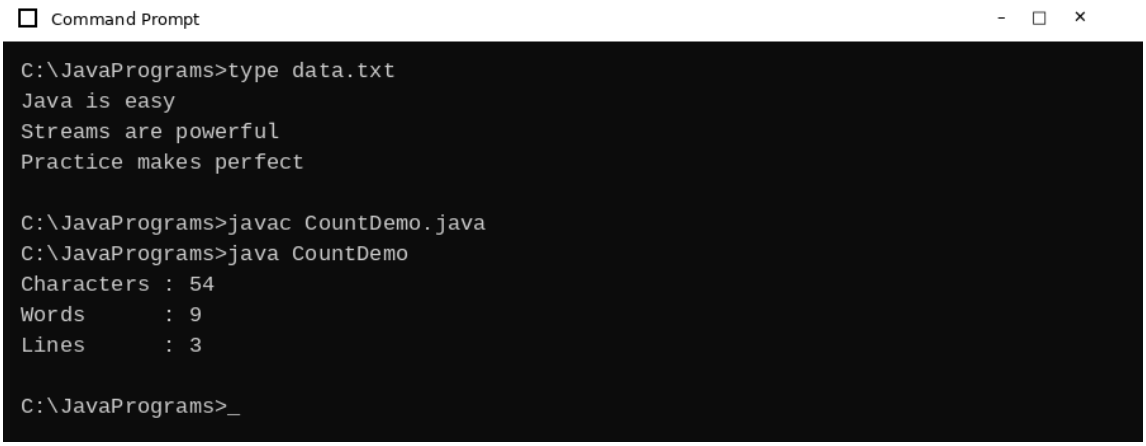
public class CountDemo {
    public static void main(String[] args) throws IOException {
        FileReader fr = new FileReader("data.txt");

        int ch, chars = 0, words = 0, lines = 0;
        boolean inWord = false;

        while ((ch = fr.read()) != -1) {
            if (ch == '\r') continue;          // ignore carriage return
            if (ch == '\n') {
                lines++;
            } else {
                chars++;
            }
            if (ch == ' ' || ch == '\n') {
                inWord = false;
            } else if (!inWord) {
                inWord = true;
                words++;
            }
        }
        fr.close();

        System.out.println("Characters : " + chars);
        System.out.println("Words      : " + words);
        System.out.println("Lines      : " + lines);
    }
}
```

Output:



Question 6

Write a Java program to demonstrate the Main Thread and display its name, priority, and state.

Program (MainThreadDemo.java):

```
public class MainThreadDemo {
    public static void main(String[] args) {
        Thread t = Thread.currentThread();

        System.out.println("Thread Name : " + t.getName());
        System.out.println("Priority    : " + t.getPriority());
        System.out.println("State       : " + t.getState());

        t.setName("MyMainThread");
        t.setPriority(8);
        System.out.println("\nAfter changing name and priority:");
        System.out.println("Thread Name : " + t.getName());
        System.out.println("Priority    : " + t.getPriority());
    }
}
```

Output:



Question 7

Write a Java program to create a thread by extending the Thread class.

Program (ExtendThreadDemo.java):

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(getName() + " : count = " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class ExtendThreadDemo {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
    }
}
```

Output:



Question 8

Write a Java program to create a thread by implementing the Runnable interface.

Program (RunnableDemo.java):

```
class MyRunnable implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            String name = Thread.currentThread().getName();
            System.out.println(name + " : count = " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class RunnableDemo {
    public static void main(String[] args) {
        MyRunnable r = new MyRunnable();
        Thread t = new Thread(r, "Worker-1");
        t.start();
    }
}
```

Output:



Question 9

Write a Java program to create multiple threads and demonstrate their concurrent execution.

Program (MultiThreadDemo.java):

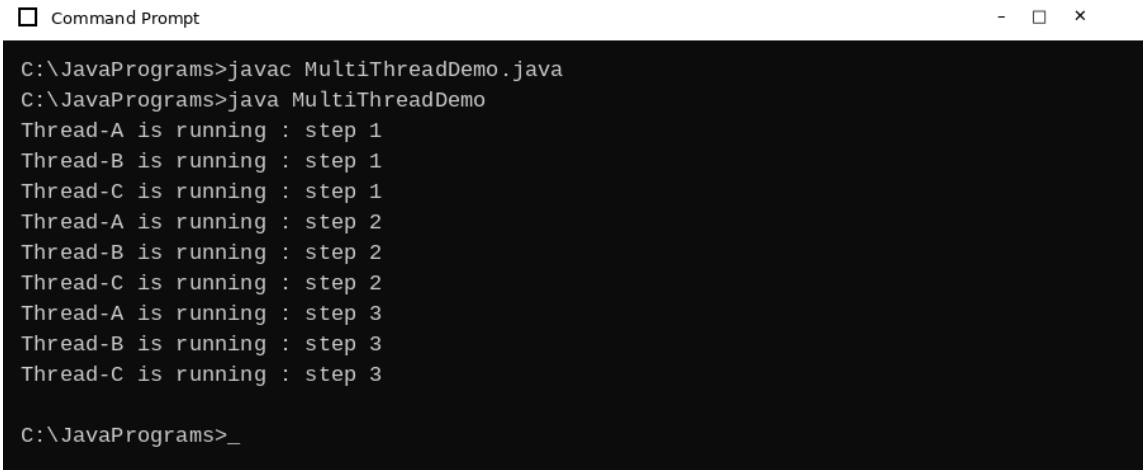
```
class Task extends Thread {
    Task(String name) {
        super(name);
    }

    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(getName() + " is running : step " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class MultiThreadDemo {
    public static void main(String[] args) throws InterruptedException {
        Task t1 = new Task("Thread-A");
        Task t2 = new Task("Thread-B");
        Task t3 = new Task("Thread-C");

        t1.start();
        Thread.sleep(50);
        t2.start();
        Thread.sleep(50);
        t3.start();
    }
}
```

Output:



Question 10

Write a Java program to demonstrate the use of isAlive() method to check whether a thread is running.

Program (IsAliveDemo.java):

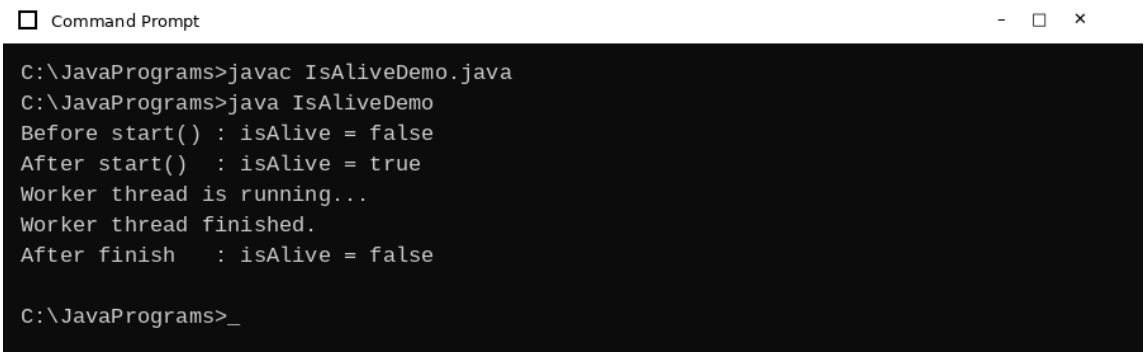
```
class Worker extends Thread {
    public void run() {
        System.out.println("Worker thread is running...");
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            System.out.println(e);
        }
        System.out.println("Worker thread finished.");
    }
}

public class IsAliveDemo {
    public static void main(String[] args) throws InterruptedException {
        Worker w = new Worker();

        System.out.println("Before start() : isAlive = " + w.isAlive());
        w.start();
        System.out.println("After start()  : isAlive = " + w.isAlive());

        w.join();
        System.out.println("After finish   : isAlive = " + w.isAlive());
    }
}
```

Output:



Question 11

Write a Java program to demonstrate the use of join() method to make one thread wait for another thread to complete.

Program (JoinDemo.java):

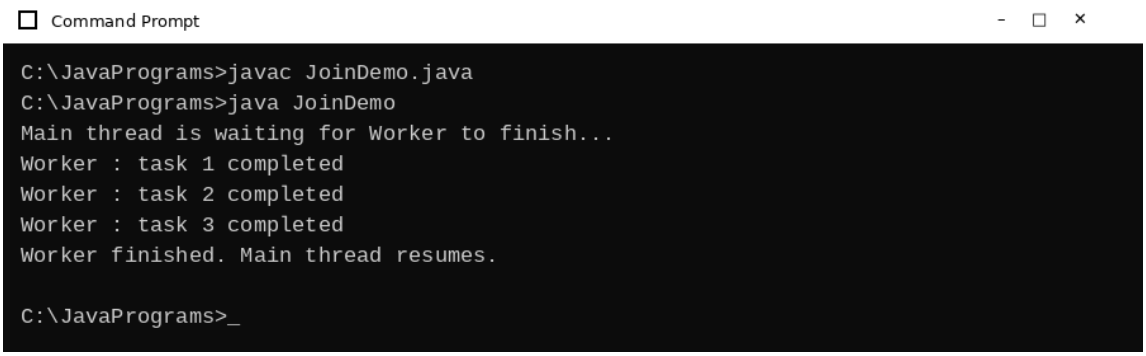
```
class Worker extends Thread {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
            System.out.println("Worker : task " + i + " completed");
        }
    }
}

public class JoinDemo {
    public static void main(String[] args) throws InterruptedException {
        Worker w = new Worker();
        w.start();

        System.out.println("Main thread is waiting for Worker to finish...");
        w.join();   // main waits here until Worker completes

        System.out.println("Worker finished. Main thread resumes.");
    }
}
```

Output:



Question 12

Write a Java program to create multiple threads and use isAlive() and join() to control their execution order.

Program (OrderControlDemo.java):

```
class Task extends Thread {
    Task(String name) {
        super(name);
    }

    public void run() {
        try {
            Thread.sleep(400);
        } catch (InterruptedException e) {
            System.out.println(e);
        }
        System.out.println(getName() + " is running");
    }
}

public class OrderControlDemo {
    public static void main(String[] args) throws InterruptedException {
        Task[] threads = new Task[3];
        for (int i = 0; i < 3; i++) {
            threads[i] = new Task("Thread-" + (i + 1));
        }

        for (Task t : threads) {
            String n = t.getName();
            t.start();
            System.out.println(n + " started, alive = " + t.isAlive());
            t.join();   // wait before starting next thread
            System.out.println(n + " finished, alive = " + t.isAlive());
        }
        System.out.println("All threads completed in order.");
    }
}
```

Output:

